

EXERCISE 07 · UNIT 2

Load imbalance and the schedule clause

Parallelising this loop is one line. Making it scale is one word.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

EXERCISE

ex07-openmp-load-balance

LANGUAGE

C, OpenMP

SUBMIT AS

<ROLLNO>.c

UNIT

2 — OpenMP and MPI

MARKS

12

OFFERING

Odd Semester 2026-27

The problem

Every row of this workload does a different amount of work. Row i performs `cost(i)` inner iterations, and `cost` grows **quadratically** with i :

```
int cost(int i) { return 1 + (i * i) / 40; }
```

At $n = 3000$, row 2999 costs about 225 000 iterations while row 0 costs one. The total work is dominated entirely by the tail.

Why the obvious answer fails

```
#pragma omp parallel for          /* default: static */  
for (int i = 0; i < n; i++) { ... }
```

The default **static** schedule splits the iteration space into equal-sized contiguous chunks. Equal in *number of iterations*, not in *work*.

On four threads, the thread that receives the last quarter of the rows does roughly **half of all the work in the program**, while the other three finish early and wait at the implicit barrier. You have bought four cores and you are running at not much better than one.

Measured on the reference machine, static gives **50 percent** efficiency; a correct schedule gives **100 percent**.

The tool

<code>schedule(static)</code>	equal chunks, decided up front, no runtime cost
<code>schedule(static, k)</code>	round-robin in chunks of k
<code>schedule(dynamic, k)</code>	threads take the next chunk of k when they finish
<code>schedule(guided)</code>	large chunks first, shrinking towards the end

SCHEDULE	OVERHEAD	USE WHEN
<code>static</code>	Lowest	Every iteration costs about the same
<code>dynamic</code>	Highest	Cost varies a lot or unpredictably
<code>guided</code>	Middle	Cost varies moderately; a good default when unsure

PITFALL · THE CHUNK SIZE IS PART OF THE ANSWER

`schedule(dynamic)` on its own means a chunk size of **one**. When an iteration is cheap, the runtime coordination to hand out a single iteration costs more than performing it, and a dynamic schedule can end up slower than a static one.

Pick a chunk large enough to amortise the bookkeeping and small enough to still balance. This is the **granularity** decision from Unit 1, in one argument.

Rules

- Do not add a `main()`. The tests provide one.
- Do not change `cost()` and do not skip work. Every implementation must perform exactly the same arithmetic; only the **assignment of rows to threads** may differ.
- `out[i]` must not depend on the thread count. A determinism check runs the tests repeatedly at 1, 2 and 4 threads.

Building and testing

```
./selfcheck.sh  
OMP_NUM_THREADS=8 ./selfcheck.sh
```

ON ASAI COMPUTE · WHERE THE EFFECT IS VISIBLE

Two cores show the difference; sixteen make it obvious.

```
bash for t in 1 2 4 8 16; do srun --cpus-per-task=16 --time=00:05:00 \ bash -c  
"OMP_NUM_THREADS=$t OMP_PROC_BIND=close ./bench 3000" done
```

Do this with `static` and with your chosen schedule, and plot both. The static curve flattens almost immediately; the balanced one keeps climbing.

How this is marked

CHECK	MARKS
Compiles cleanly	1
Uses OpenMP	1
Public tests	2
Hidden tests	2
Deterministic across thread counts	2
Balances the load (parallel efficiency at least 70%)	4

A static schedule scores zero on the last row. That is the exercise.

Submitting

AIE23001.c